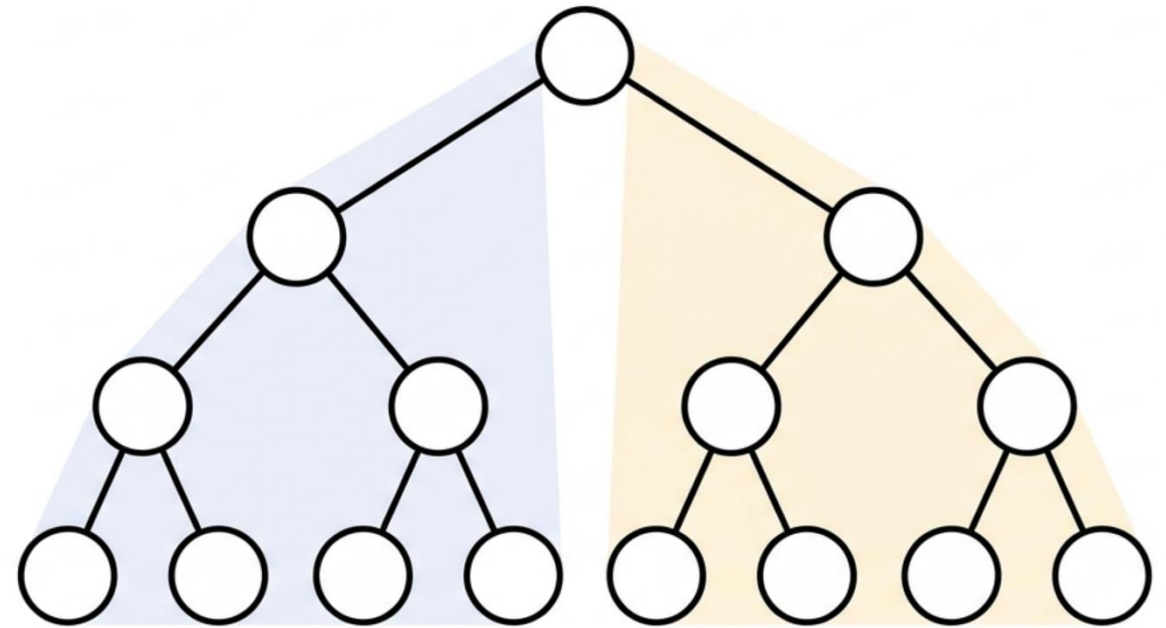


Árboles de búsqueda Binario (BST)

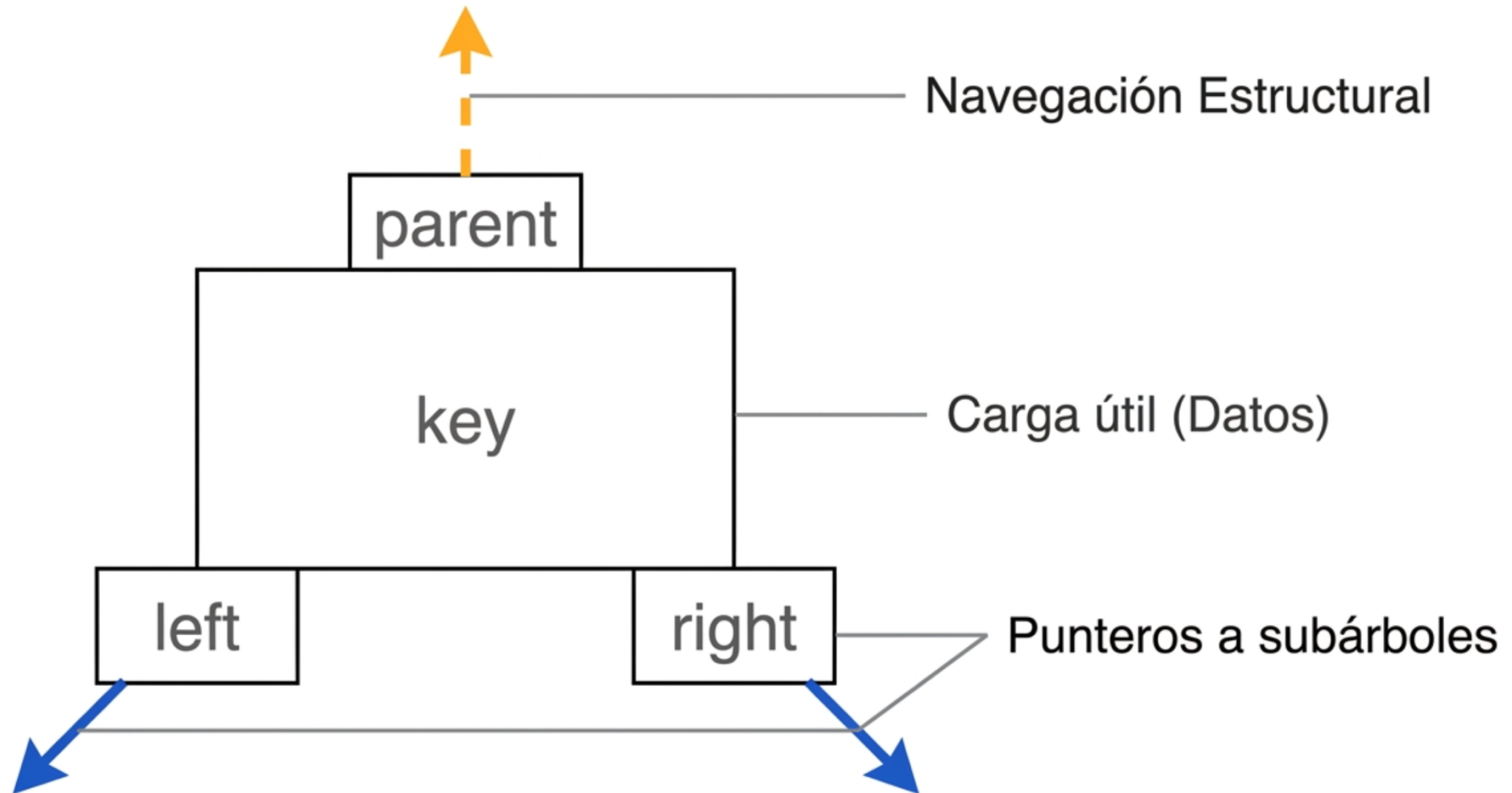


Objetivos

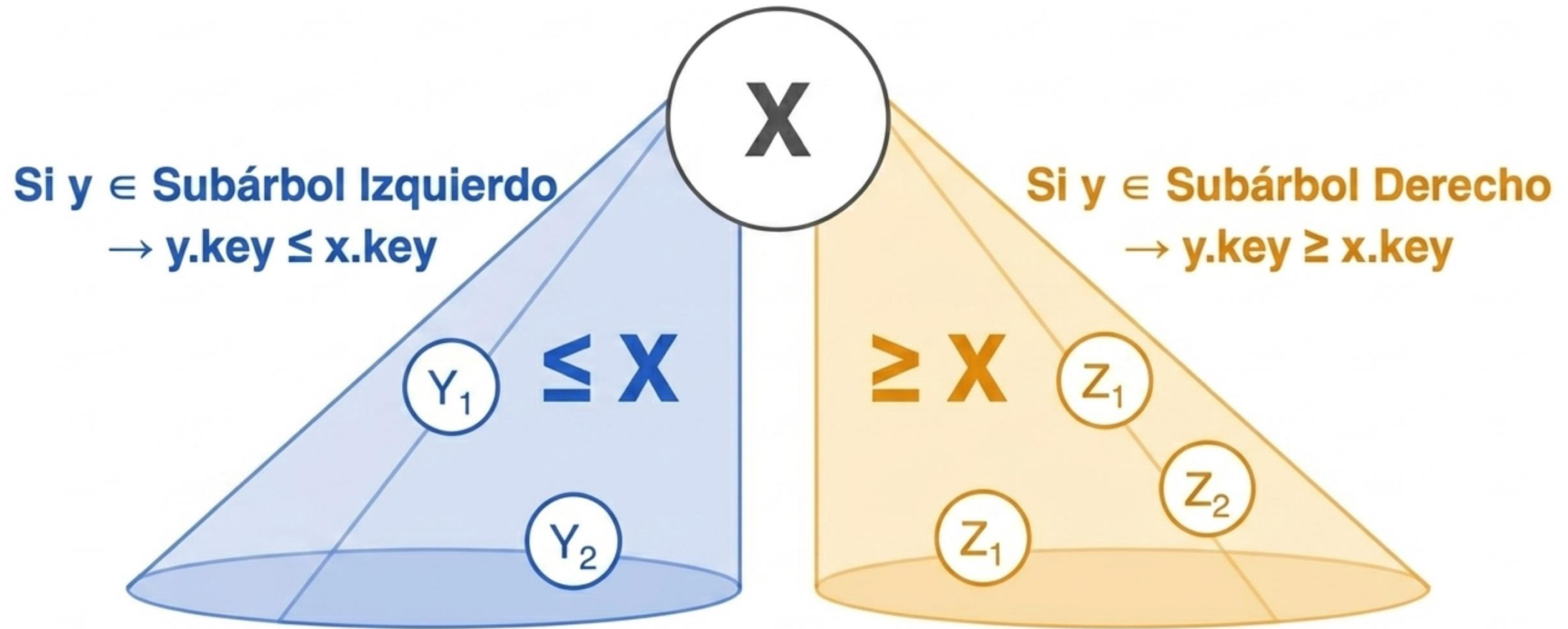
- Comprender la invariante topológica del BST y aplicar el orden estricto débil para implementaciones de conjuntos dinámicos.
- Diseñar la estructura en C++ moderno (C++17/20) utilizando el paradigma RAII (`std::unique_ptr`) para prevenir fugas de memoria, separando el contrato (Interfaz) de la implementación.
- Implementar la operación de eliminación (TREE-DELETE) aplicando la subrutina TRANSPLANT sin violar el ciclo de vida de los punteros inteligentes.



Topología de un BST

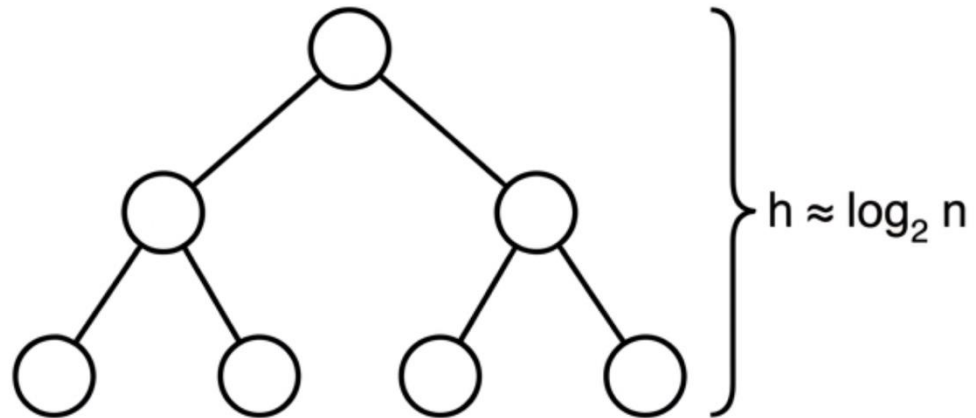


Invariante de un BST



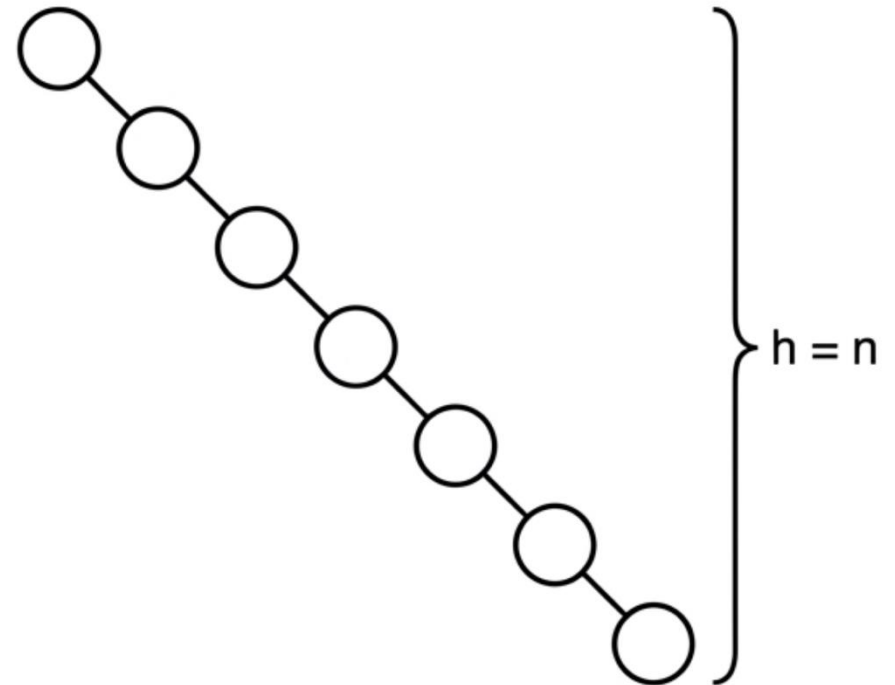
Rendimiento de un BST

Árbol Balanceado



Complejidad: $O(\log n)$

Árbol Degenerado (Lista Enlazada)

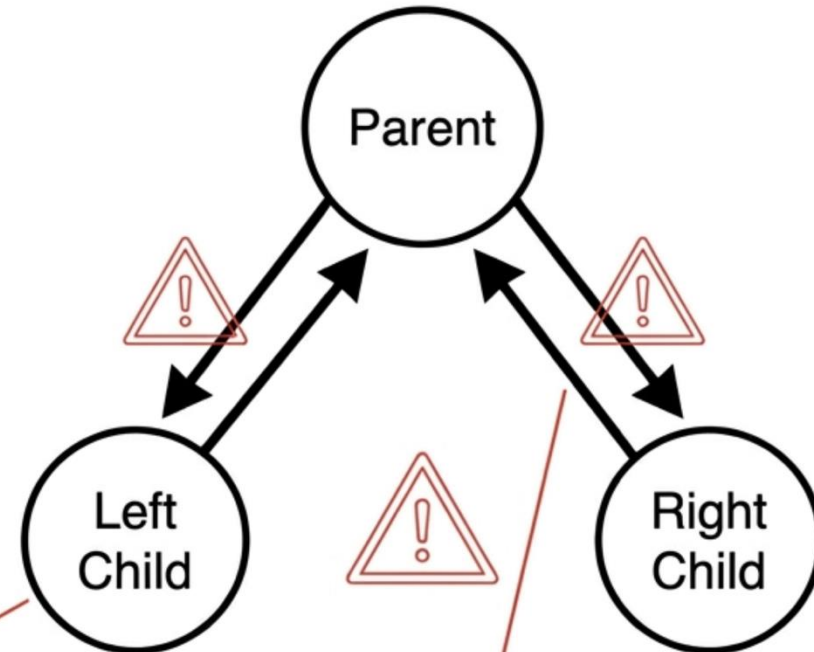


Complejidad: $O(n)$

Manejo de la memoria

- Gestión manual (new / delete)
- Riesgo de fugas de memoria (Memory leaks)
- Referencias circulares

Si el padre se destruye sin liberar a los **hijos**, estos se convierten en huérfanos en el heap sin heap (Memory Leak).



Punteros **fuertes** en ambas direcciones crean ciclos de referencia irresolubles.

Manejo de la memoria



Universidad del
Rosario

Escuela de Ciencias
e Ingeniería



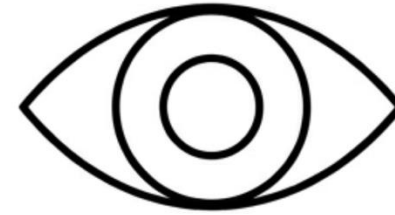
MACC

std::unique_ptr
(Propiedad / Ciclo de vida)



Gestión determinista.
La destrucción fluye hacia abajo.

Raw Pointer
(Observación Estructural)



Navegación segura hacia la raíz
(sucesor/predecesor).
No controla la memoria.

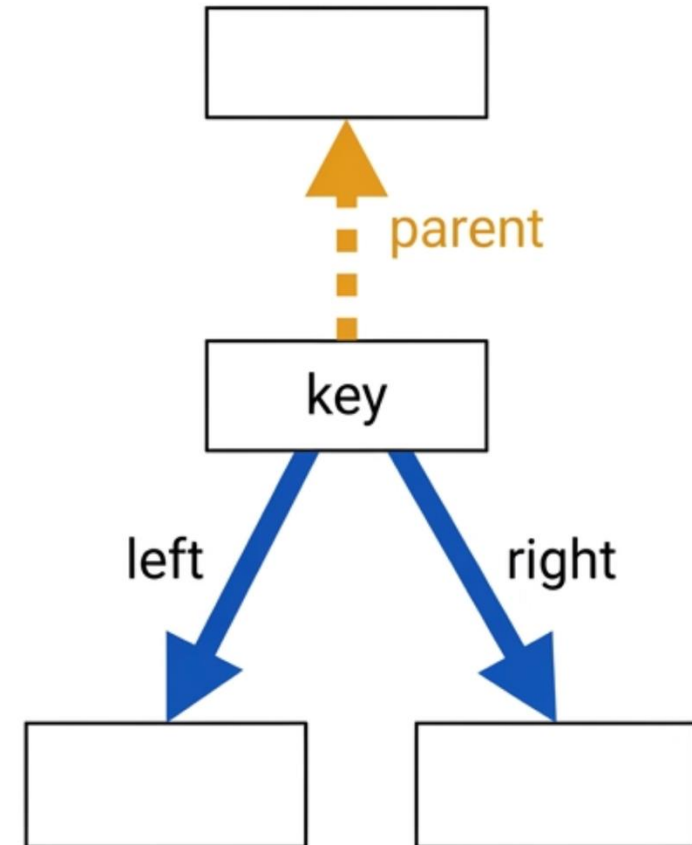
Implementación

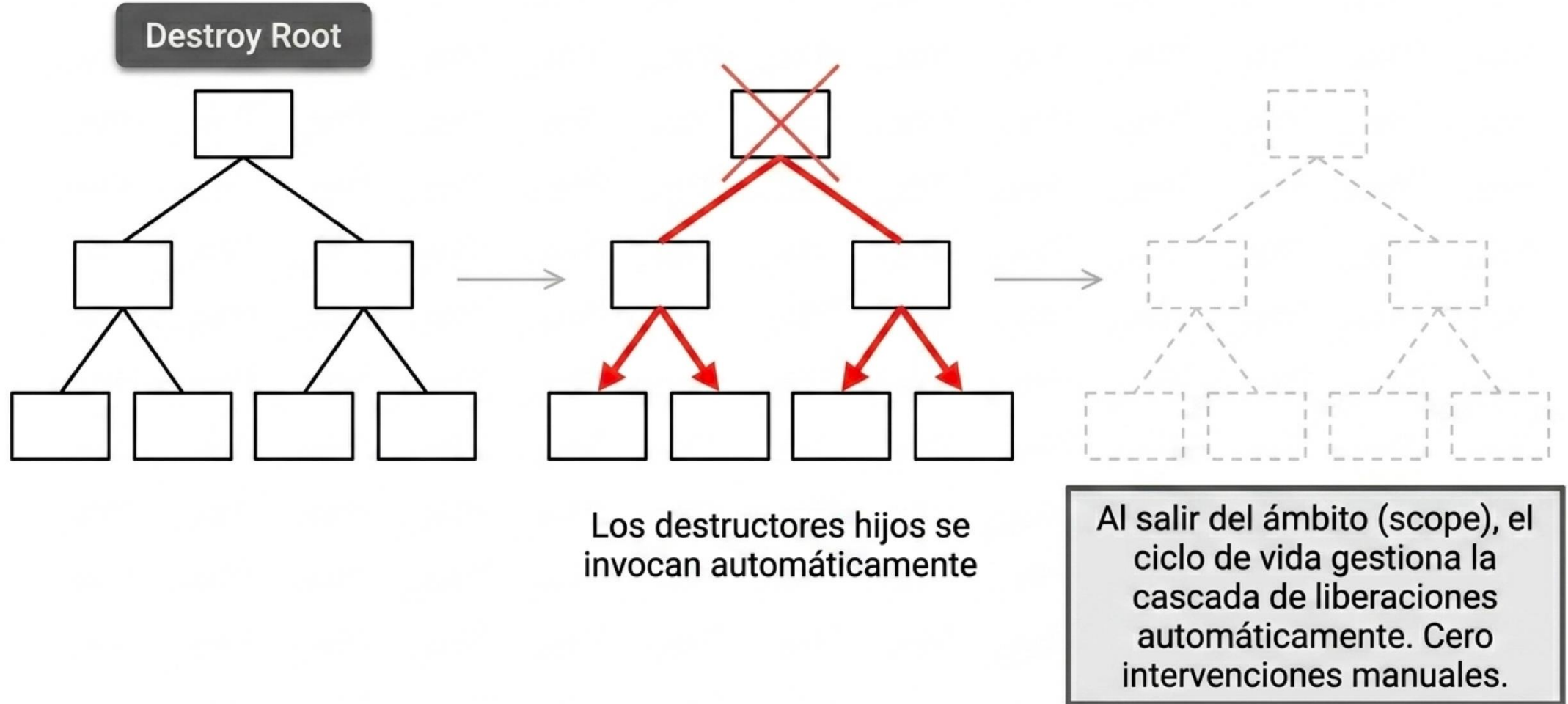


```
template <typename T>
struct BSTNode {
    T key;
    BSTNode* parent;
    std::unique_ptr<BSTNode> left;
    std::unique_ptr<BSTNode> right;
    explicit BSTNode(T k, BSTNode* p = nullptr)...
};
```

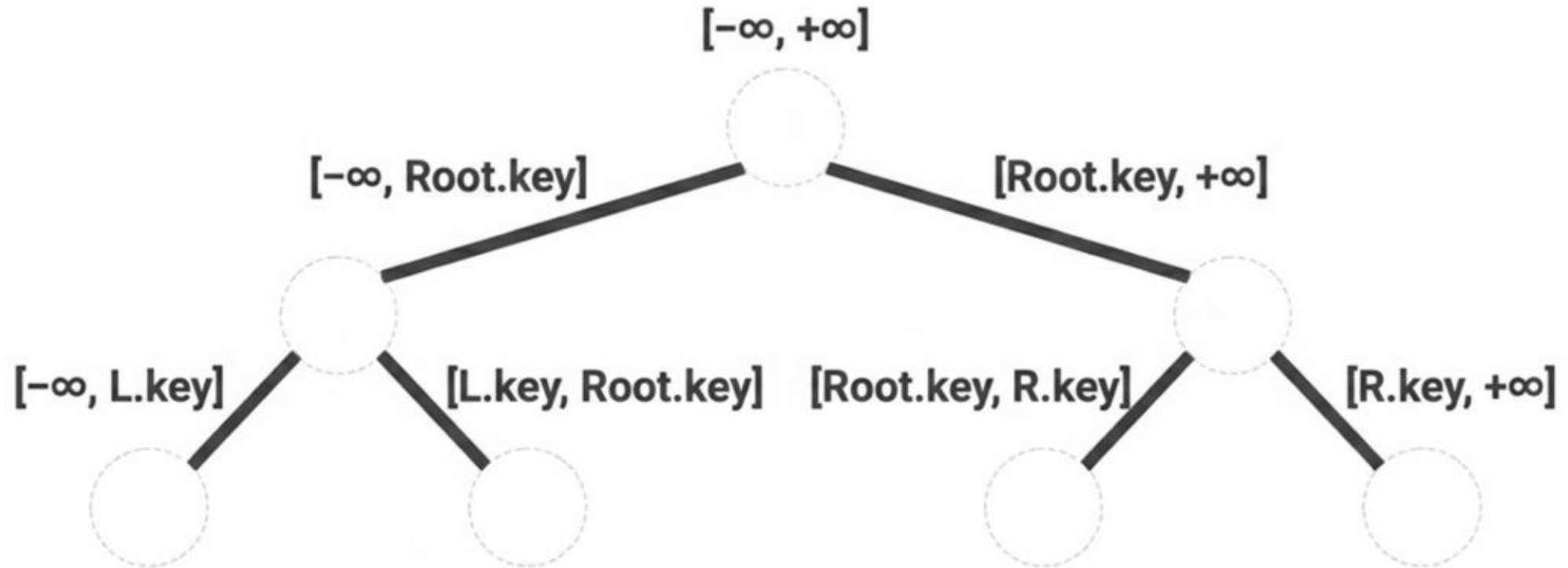
Evita conversiones
implícitas

Manejo determinista
del ciclo de vida





Invariante del BST



Para ser válido, cada nodo debe existir **estrictamente** dentro del rango heredado de sus ancestros.

Implementación

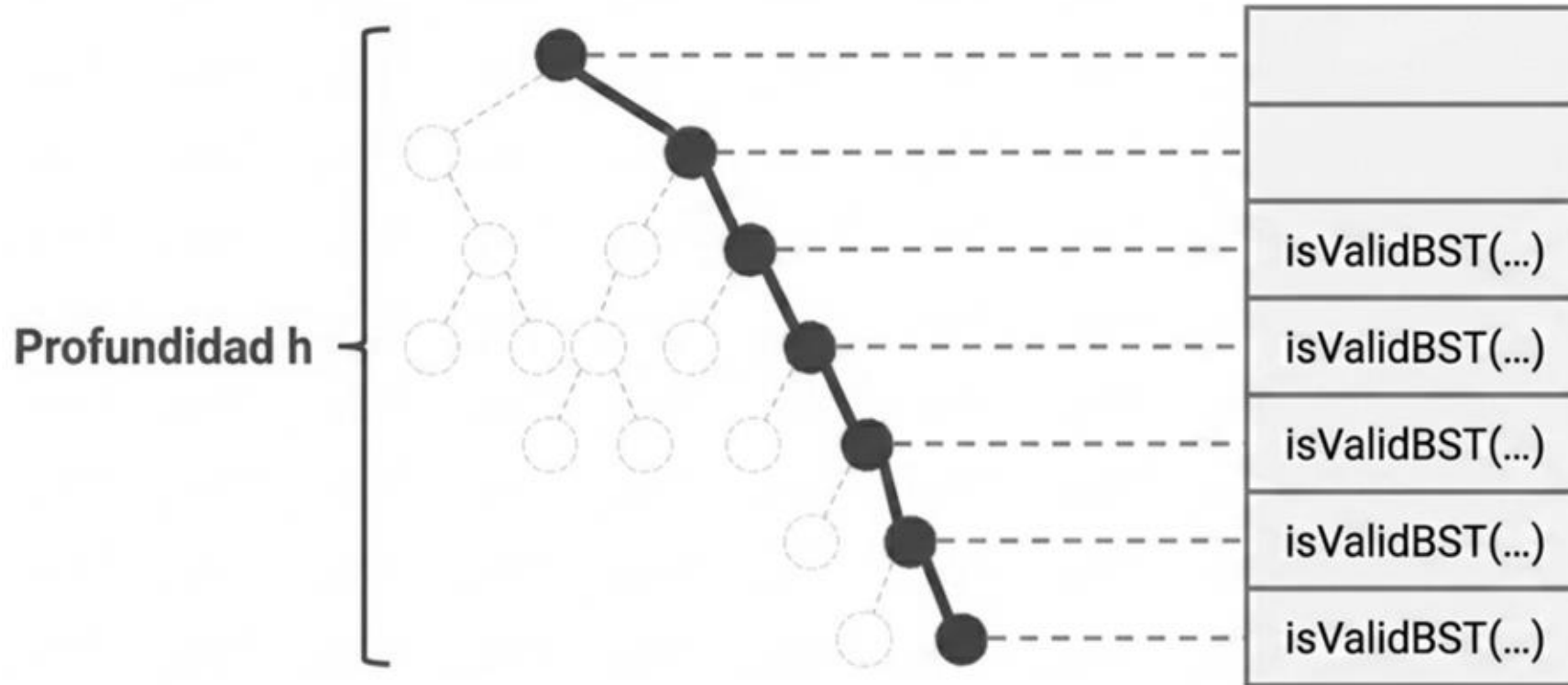
```
bool isValidBST(const BSTNode<T>* node, T min_val, T max_val) {  
    if (node == nullptr) return true;  
  
    if (node->key < min_val || node->key > max_val) {  
        return false;  
    }  
  
    return isValidBST(node->left.get(), min_val, node->key) &&  
           isValidBST(node->right.get(), node->key, max_val);  
}
```

Vacuamente cierto
(Caso base)

Violación de la
invariante

Propagación de
límites dinámicos
 $O(h)$

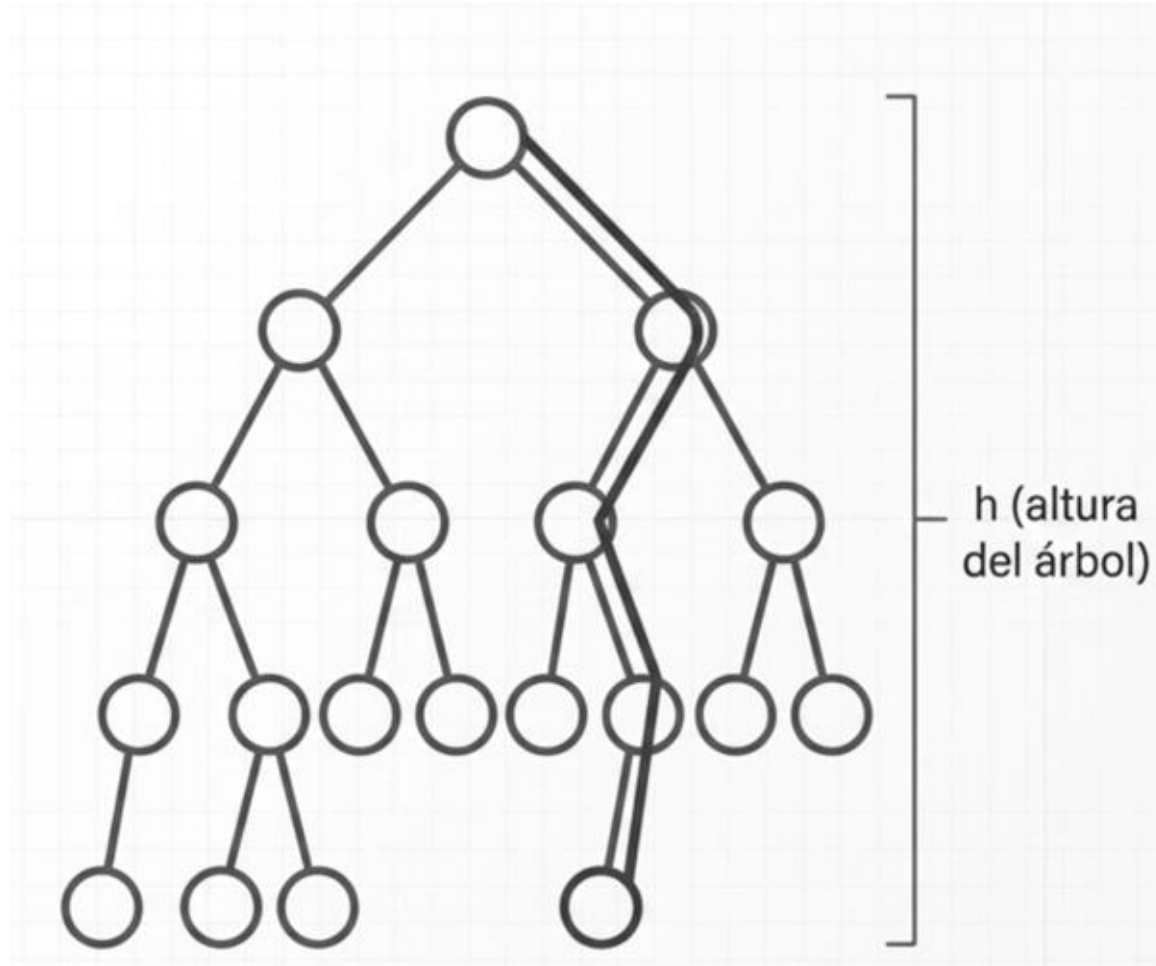
Complejidad espacial



Complejidad Temporal: $O(n)$ - Cada nodo es visitado exactamente una vez.

Complejidad Espacial: $O(h)$ - La memoria auxiliar depende enteramente de la pila de llamadas (*Call Stack*), determinada por la altura del árbol.

Búsquedas



El descenso simple acota el tiempo de ejecución asintótico a $O(h)$

Los nodos encontrados durante la evaluación matemática forman un único camino hacia abajo. El número de nodos examinados está acotado por $h + 1$.

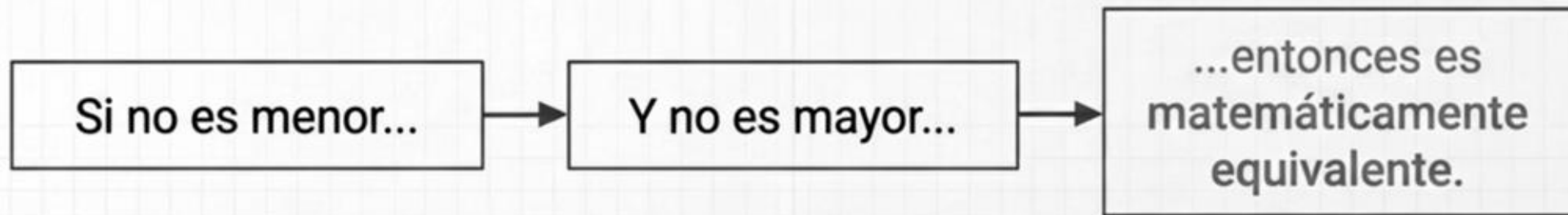
Cota superior asintótica: $O(h)$

Implementación compatible con STL



Para alinear la estructura con la STL, se prohíbe el uso del operador de igualdad ($==$). La equivalencia topológica se evalúa basándose exclusivamente en el operador de menor que ($<$).

$$a \equiv b \Leftrightarrow \neg(a < b) \wedge \neg(b < a)$$



Prevención de copias profundas



```
template <typename T>  
const BSTNode<T>* tree_search(const BSTNode<T>* x, const T& k) {
```

Observador constante al nodo.
Garantiza la inmutabilidad de la estructura.

Paso por referencia constante.
Previene copias profundas **O(n)** en la memoria cuando T es un tipo de dato complejo como `std::string`.

Prevención de copias profundas

```
while (x != nullptr) {  
    if (k < x->key) {  
        x = x->left.get();  
    }  
}
```

Despliegue iterativo.
Complejidad espacial
garantizada en **$O(1)$** .

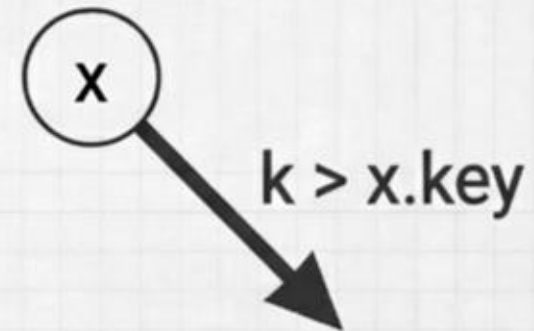
Extracción del puntero crudo
desde `std::unique_ptr`. Permite
el descenso sin alterar la propiedad
de memoria (ownership).

Prevención de copias profundas



```
else if (x->key < k) {  
    else if (x->key < k) {  
        x = x->right.get();  
    }  
}
```

Cumplimiento del invariante de Cormen. Si la clave del nodo actual es estrictamente menor al objetivo k , el algoritmo restringe su búsqueda al subárbol derecho.



Prevención de copias profundas



```
else {  
    return x;  
}  
}
```

```
return nullptr;
```

Equivalencia matemática
alcanzada: $!(k < x \rightarrow \text{key}) \ \&\&$
 $!(x \rightarrow \text{key} < k)$. Retorna el nodo
exitosamente.

Condición de fallo. Se alcanzó
una hoja nula ($O(h)$) y k no
pertenece al conjunto S .

BST robuzto



Pilar 1: Matemáticamente Riguroso

Conserva la cota superior asintótica $O(h)$ dictada por Cormen.

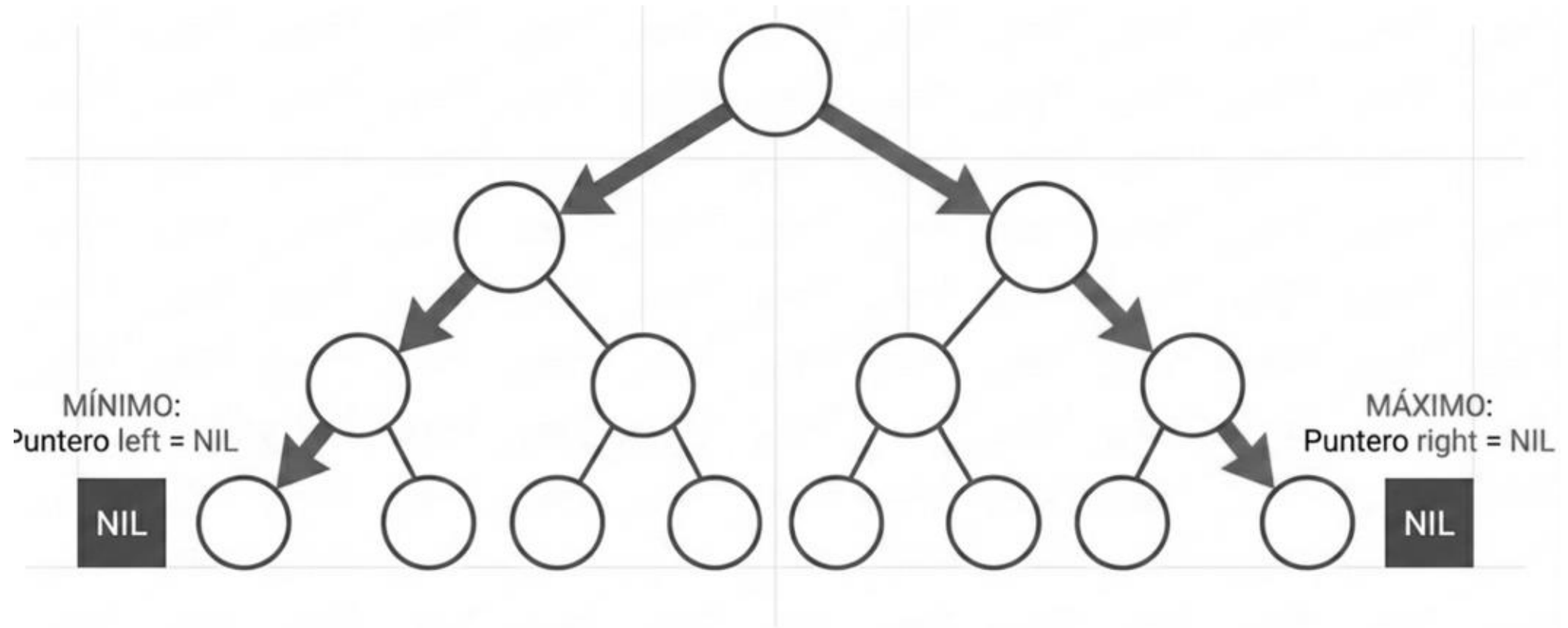
Pilar 2: Memoria Óptima

Neutraliza el riesgo de Stack Overflow con un despliegue iterativo $O(1)$ y uso seguro de punteros `get()`.

Pilar 3: Compatible STL

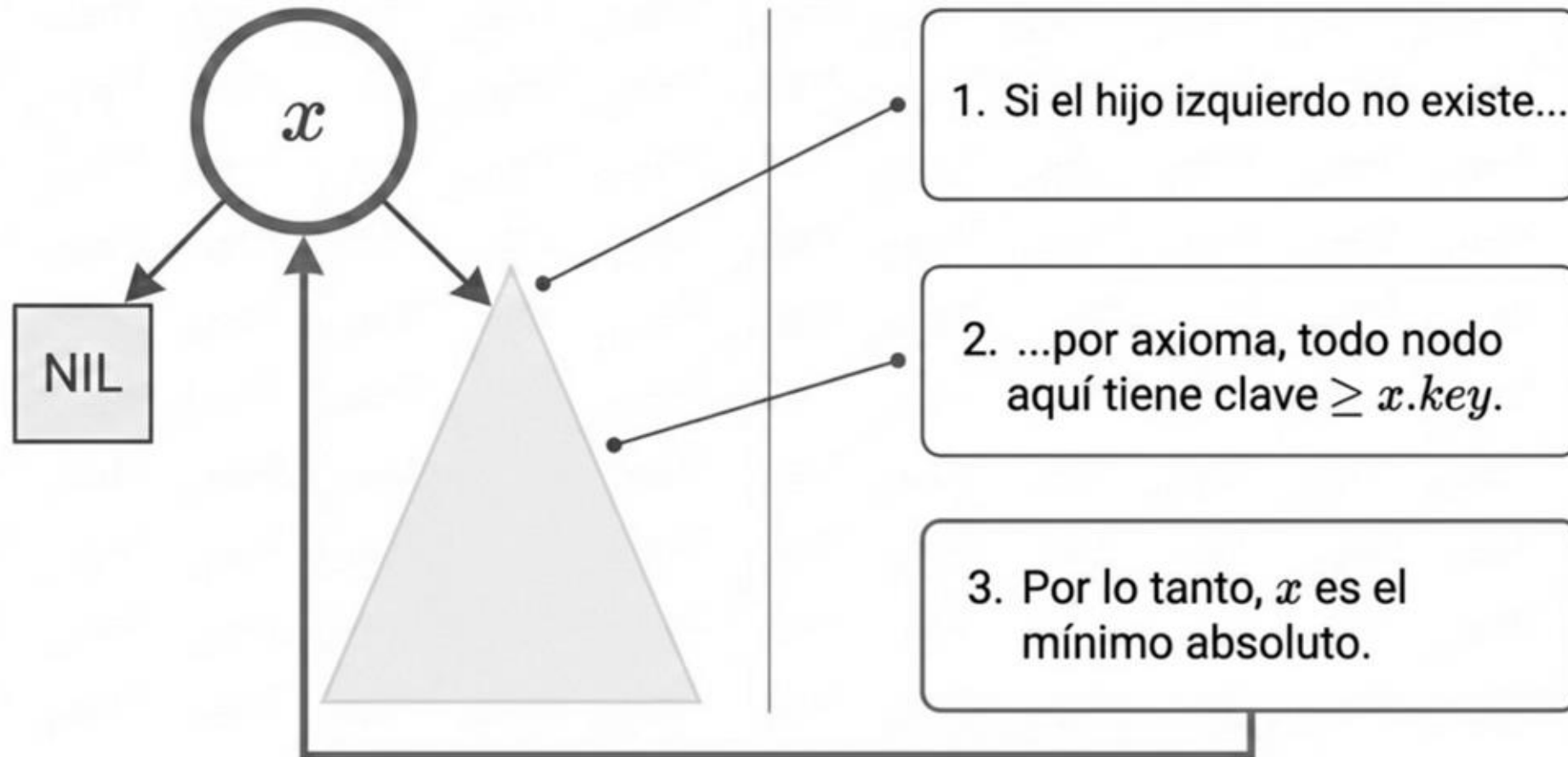
Aplica el axioma de orden estricto débil, operando independientemente del operador de igualdad.

Localización de extremos

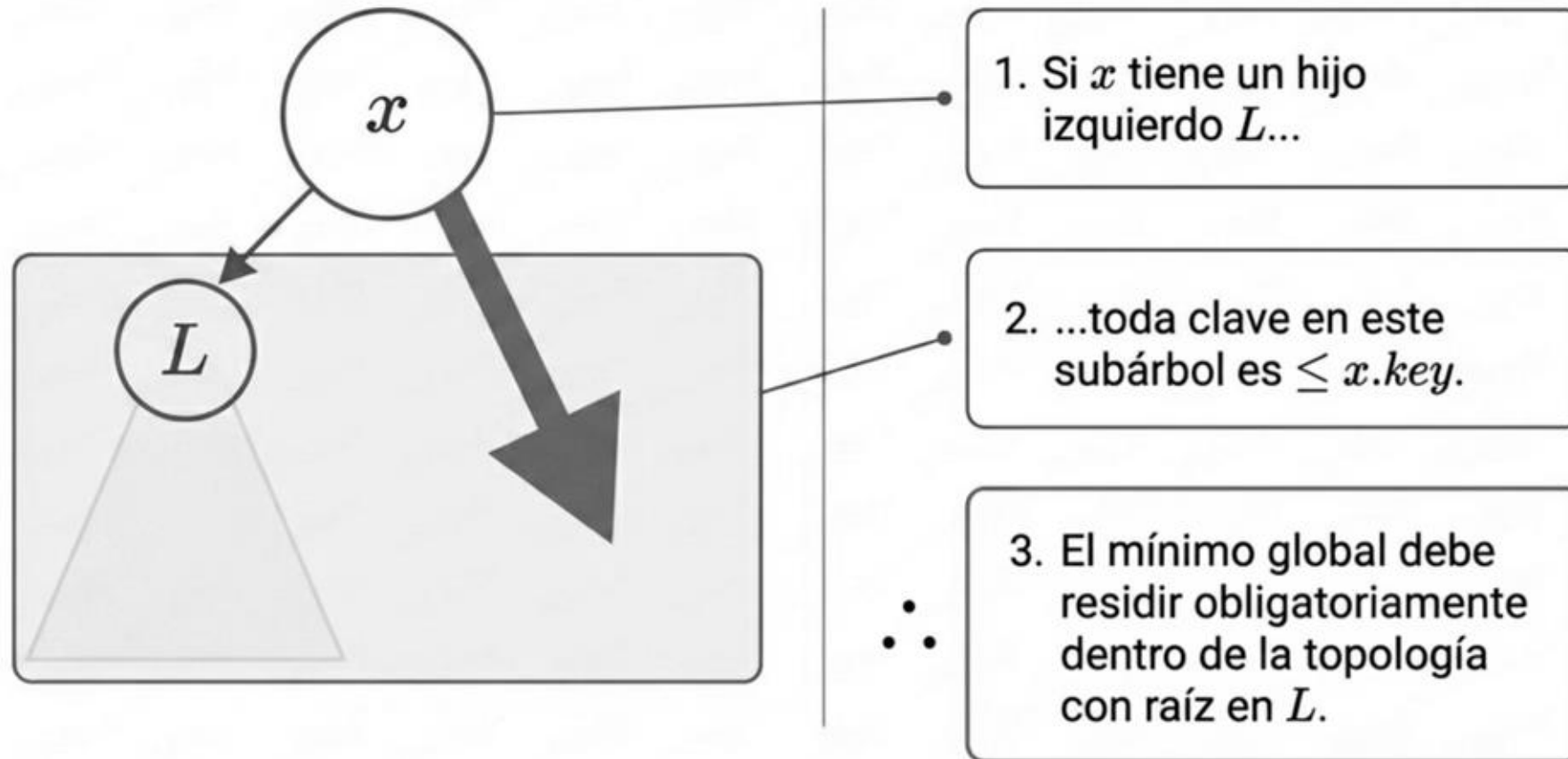


El elemento mínimo (o máximo) es el nodo alcanzado al seguir exclusivamente punteros izquierdos (o derechos) hasta encontrar un puntero vacío.

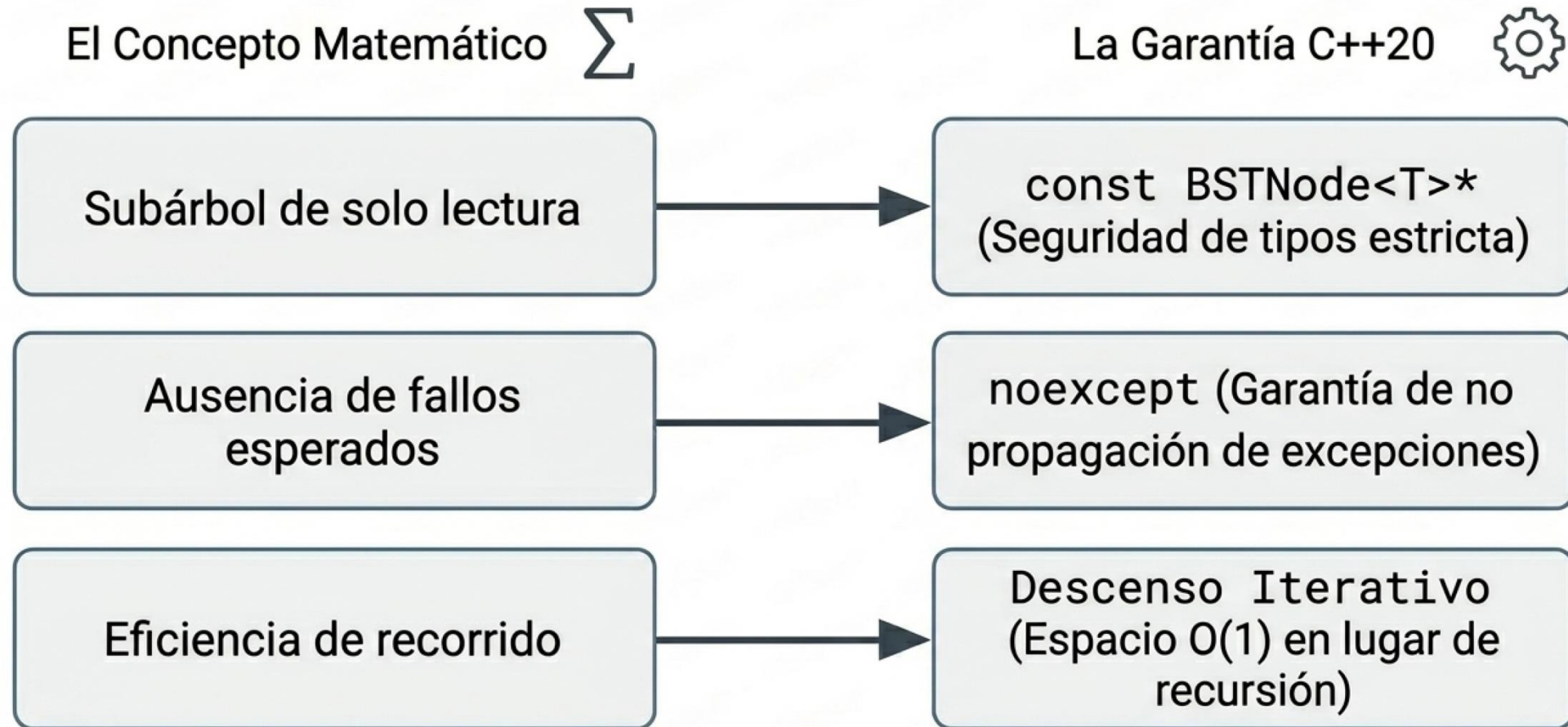
Localización de extremos



Localización de extremos



Localización de extremos



Localización de extremos

```
template <typename T>
const BSTNode<T>* tree_minimum(const BSTNode<T>* node) noexcept {
    if (node == nullptr) return nullptr;
    while (node->left != nullptr) {
        node = node->left.get();
    }
    return node;
}
```

Inmutabilidad garantizada:
el observador no puede
alterar la topología.

Garantía de rendimiento
y seguridad.

Descenso estricto iterativo.
Evita el desbordamiento de
la pila (Stack Overflow).

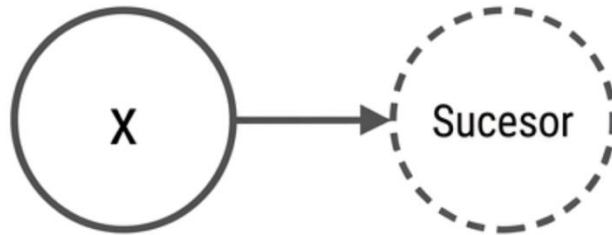
Localización de extremos

```
template <typename T>
const BSTNode<T>* tree_maximum(const BSTNode<T>* node) noexcept {
    if (node == nullptr) return nullptr;
    while (node->right != nullptr) {
        node = node->right.get();
    }
    return node;
}
```

Principio de Simetría: La misma arquitectura iterativa de seguridad $O(1)$, reflejada espacialmente hacia el borde derecho de la topología.

Recorridos

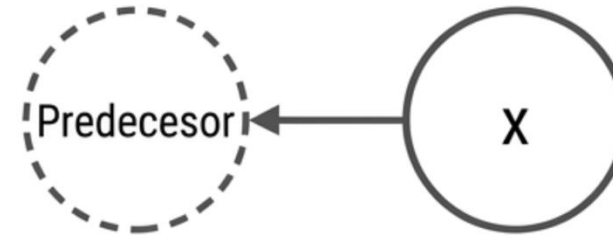
El Sucesor de x



Es el nodo con la clave más pequeña que cumple:

$$\text{clave} > x.\text{key}$$

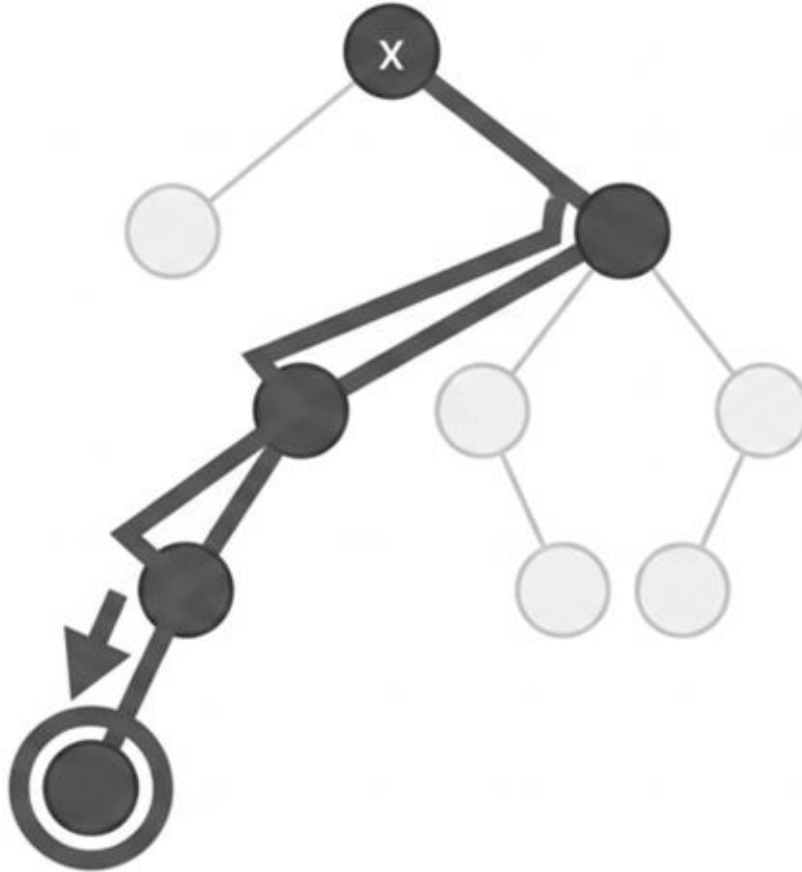
El Predecesor de x



Es el nodo con la clave más grande que cumple:

$$\text{clave} < x.\text{key}$$

Recorridos

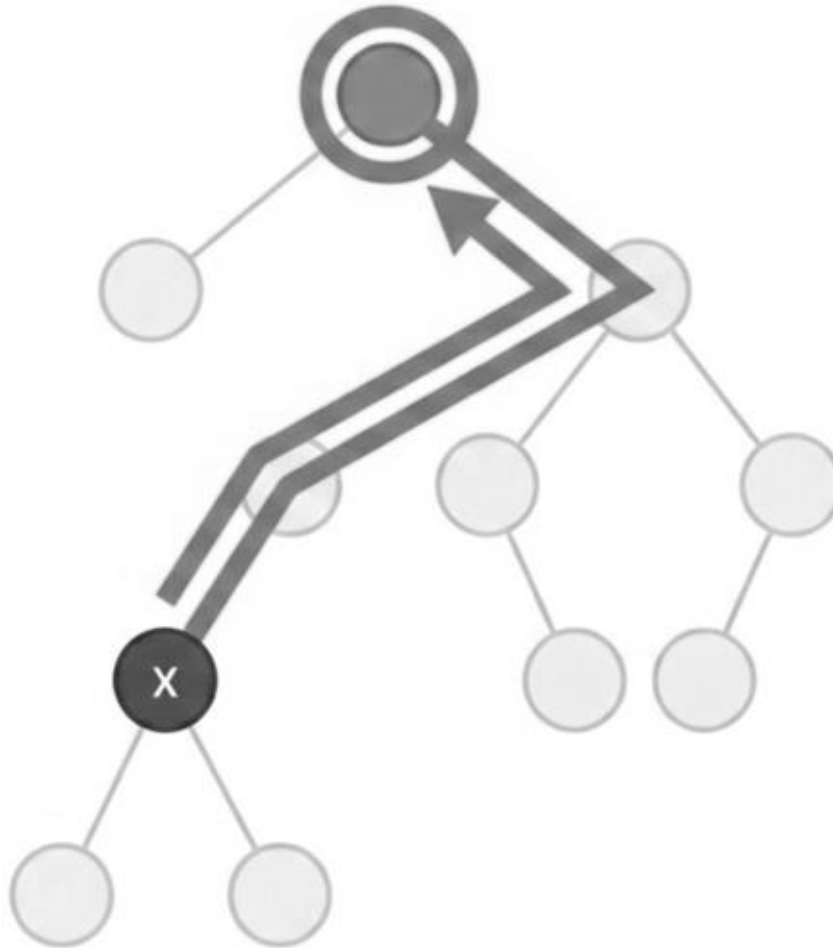


Caso 1: El Descenso Topológico

Si el subárbol derecho de x no está vacío ($x.\text{right} \neq \text{NIL}$).

- El sucesor es estrictamente el nodo mínimo del subárbol derecho.
- Representa un paso a la **derecha**, seguido de un descenso continuo hacia la izquierda.

Recorridos



Caso 2: El Ascenso Topológico

Si el subárbol derecho de x está vacío ($x.\text{right} == \text{NIL}$).

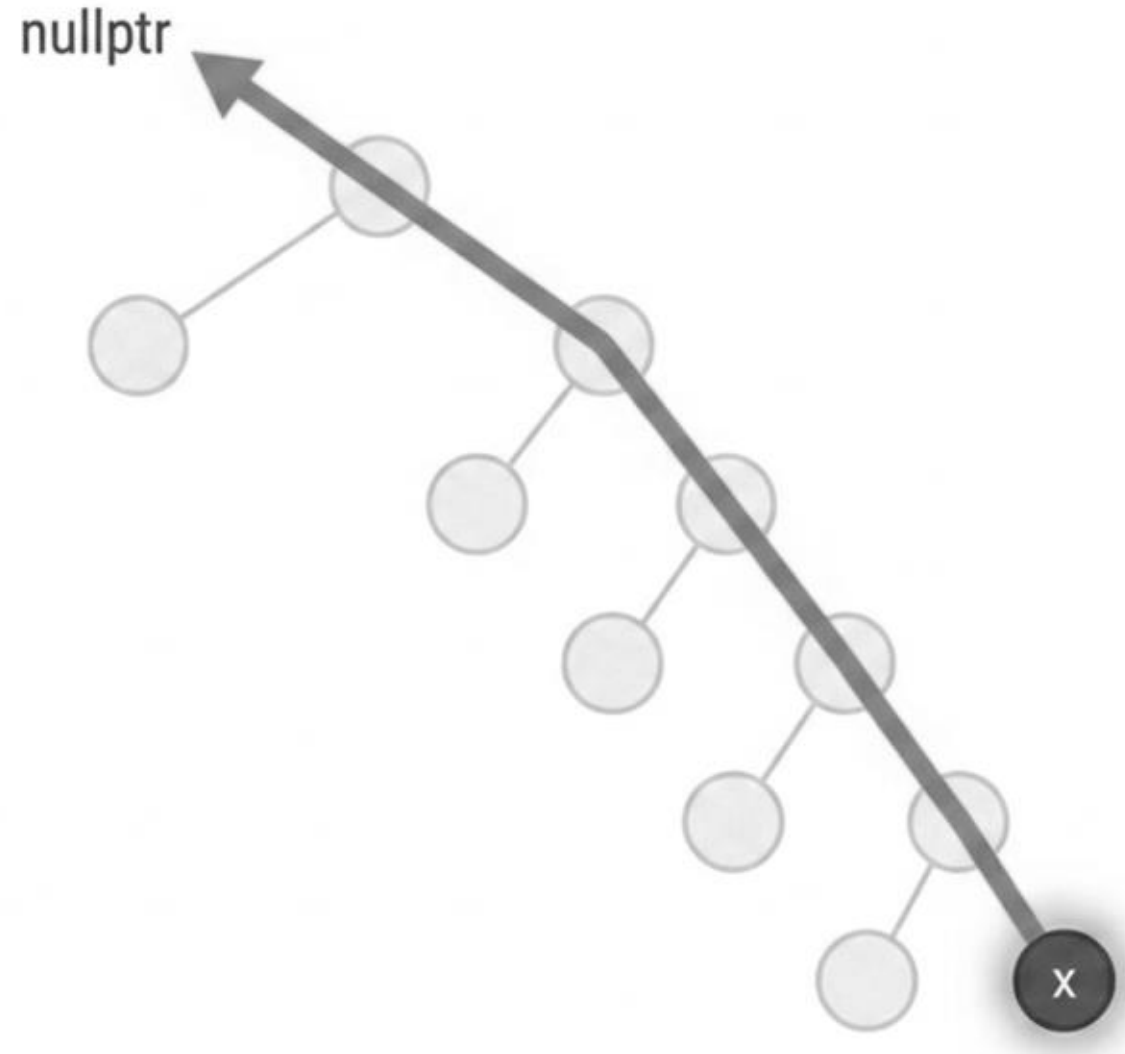
- Se inicia un rastreo ascendente por el linaje del árbol.
- El sucesor es el ancestro más bajo (lowest ancestor) cuyo hijo izquierdo es también un ancestro de x .
- Visualmente: Subir hasta encontrar el primer giro a la derecha en la topología inversa.

Recorridos

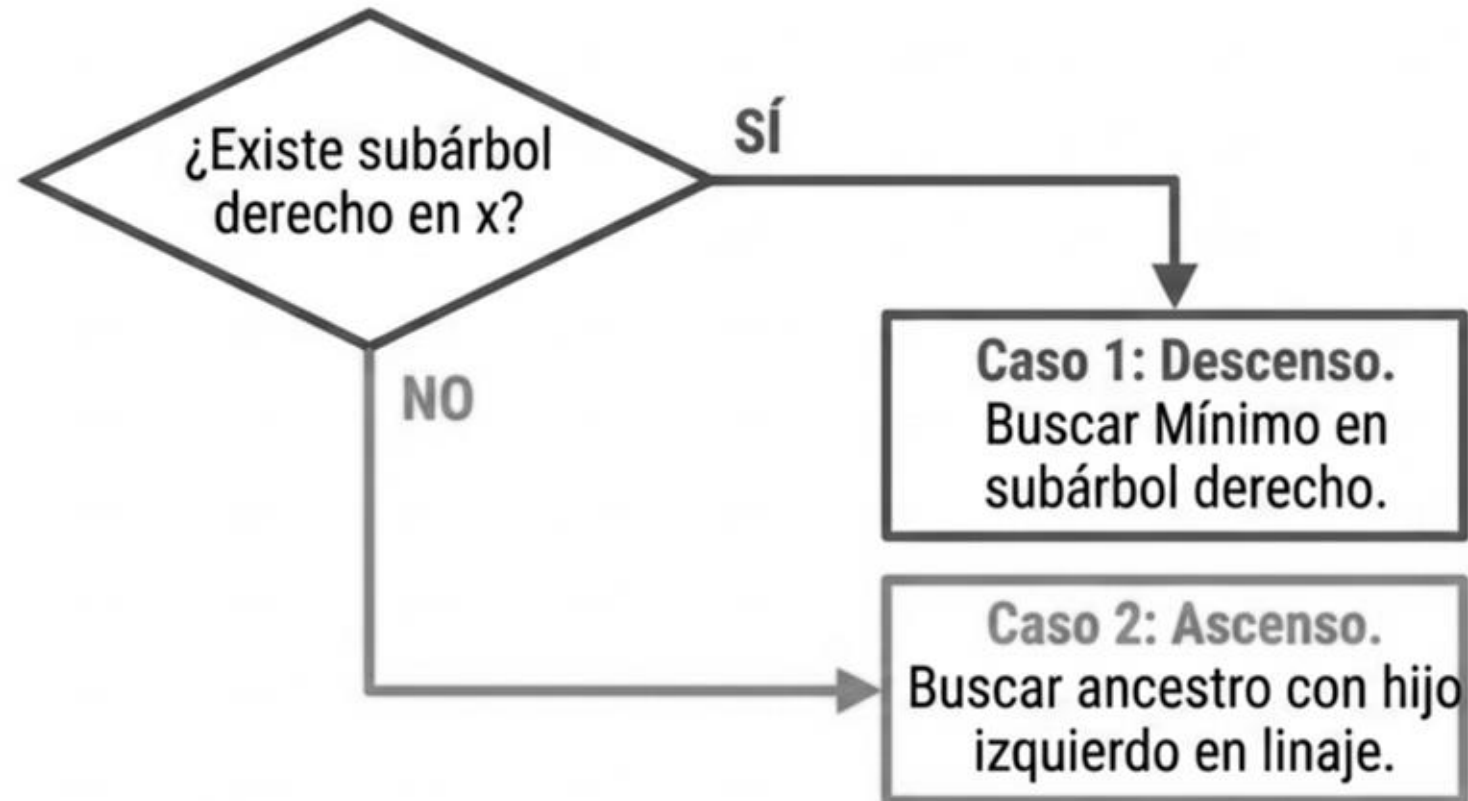
El Límite Estructural (Máximo Global)

Si el ascenso llega a la raíz sin cumplir la condición.

- x es el máximo global del árbol.
- Por definición, no existe un sucesor.
- Retorna valor
- Retorna valor nulo (nullptr).



Recorridos



El algoritmo unificado se basa enteramente en esta bifurcación estructural.

Implementación

```
template <typename T>
const BSTNode<T>* tree_successor(const BSTNode<T>* x) noexcept {
    if (x == nullptr) {
        return nullptr;
    }
    // ...
}
```

Uso del puntero de observación
garantiza seguridad (solo lectura).

Operación de navegación pura
que no lanza excepciones.

Validación estructural inicial.

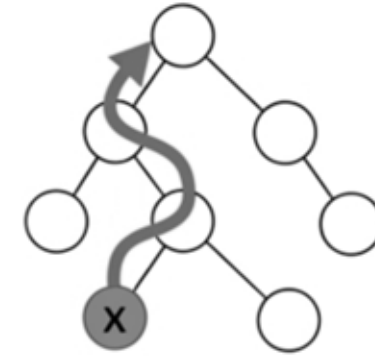
Implementación

```
// Caso 1: El subárbol derecho existe.  
if (x->right != nullptr) {  
    return tree_minimum(x->right.get());  
}
```



- Delega la responsabilidad a la subrutina `tree_minimum`.
- Mantiene la eficiencia asintótica temporal $O(h)$ mediante una abstracción limpia.

Implementación



- Los punteros iteran hacia arriba rastreando linaje.
- El bucle se rompe exactamente en el ancestro cuyo hijo izquierdo contiene a x.

```
// Para a punteros de Ascenso Topológico
const BSTNode<T>* y = x->parent;

while (y != nullptr && x == y->right.get()) {
    x = y;           // El hijo sube al padre
    y = y->parent;    // El padre sube al abuelo
}

return y; }
```

Simetría estructural del BST

Sucesor

- Si hay hijo derecho -> Buscar Mínimo.
- Si no -> Ascender por linaje Izquierdo.

Predecesor

- Si hay hijo izquierdo -> Buscar Máximo.
- Si no -> Ascender por linaje Derecho.

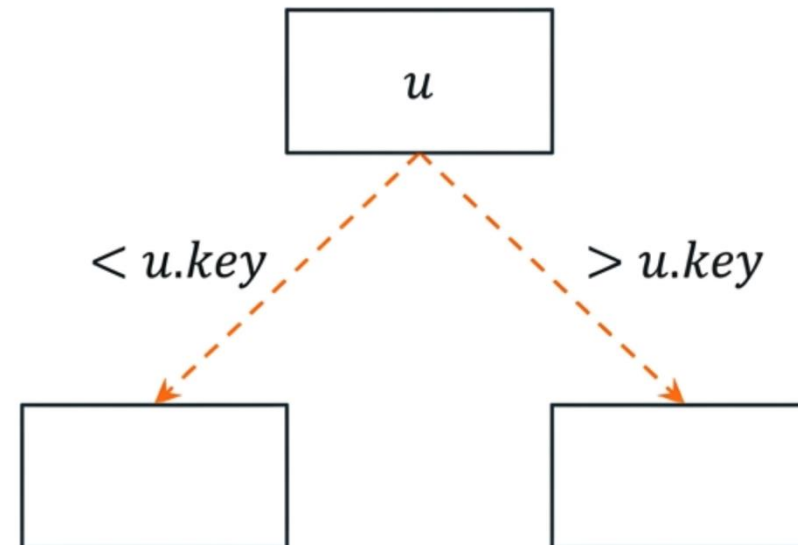
Operaciones de mutación



Toda mutación debe preservar estrictamente la invariante topológica

Invariante de Búsqueda: Para cualquier nodo u , las claves en el subárbol izquierdo son estrictamente menores, y las del derecho son mayores.

Restricción Moderna (Orden Estricto Débil): A diferencia de los modelos clásicos, en C++ moderno se rechazan las claves duplicadas para garantizar conjuntos puros y comportamiento determinista.

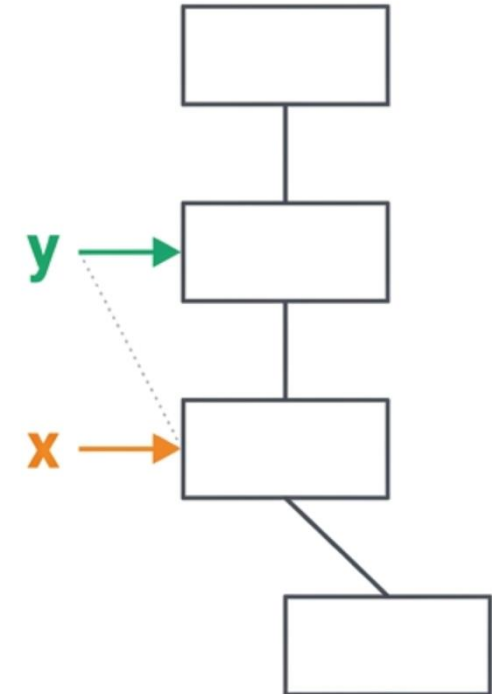


Operaciones de mutación



A diferencia de la búsqueda simple (TREE-SEARCH), la inserción altera la topología. El algoritmo no solo debe encontrar un espacio vacío (NIL), sino recordar cómo llegó a él.

- **x (Explorador)**: Traza el camino descendente hasta alcanzar NIL.
- **y (Rastro)**: Sigue un paso por detrás. Cuando x colapsa en NIL, y captura físicamente al futuro nodo padre.

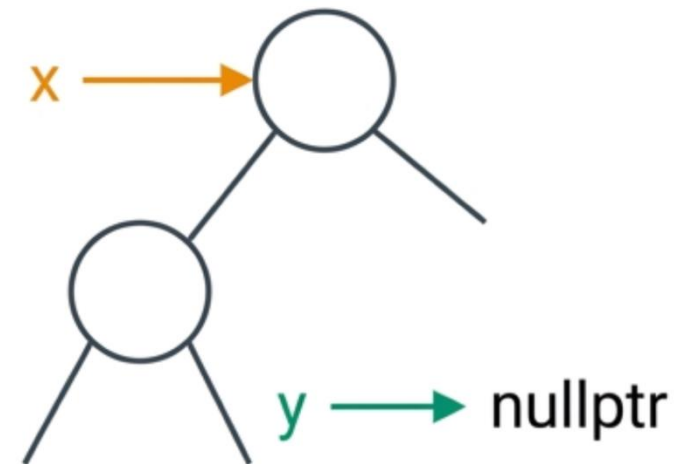
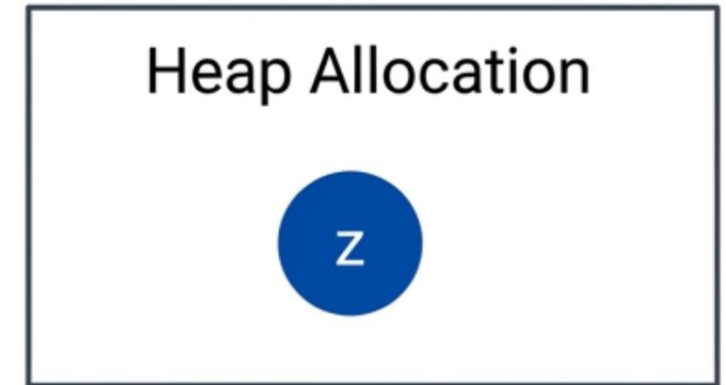


Operaciones de mutación

```
template <typename T>
void tree_insert(std::unique_ptr<BSTNode<T>>&
    root, const T& k) {
    // 1. Asignación topológica aislada en el heap
    auto z = std::make_unique<BSTNode<T>>(k);
```

```
    BSTNode<T>* y = nullptr;
    BSTNode<T>* x = root.get();
```

El nodo z es propiedad exclusiva del marco de la función temporalmente. x inicia como observador de la raíz.

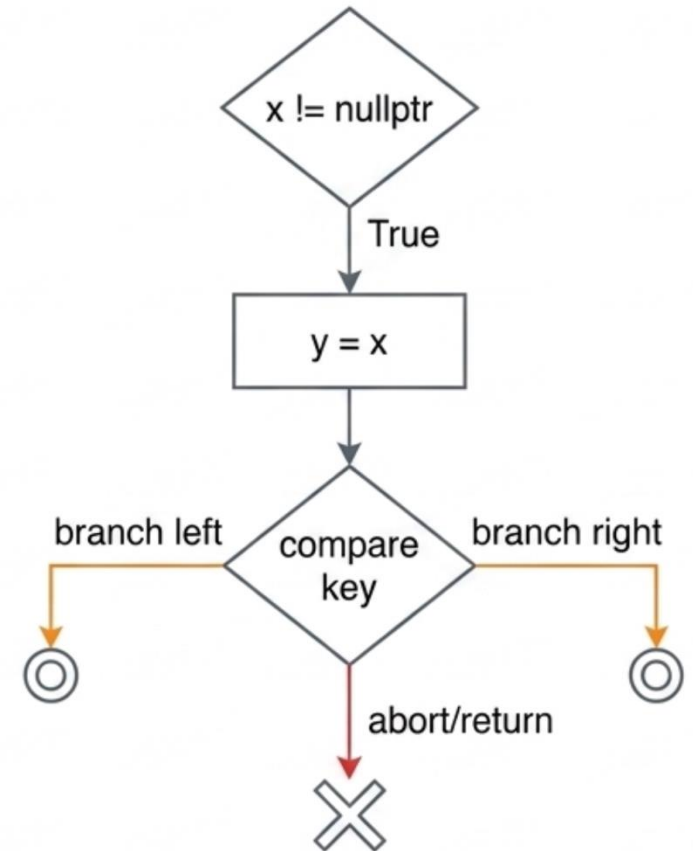


Operaciones de mutación



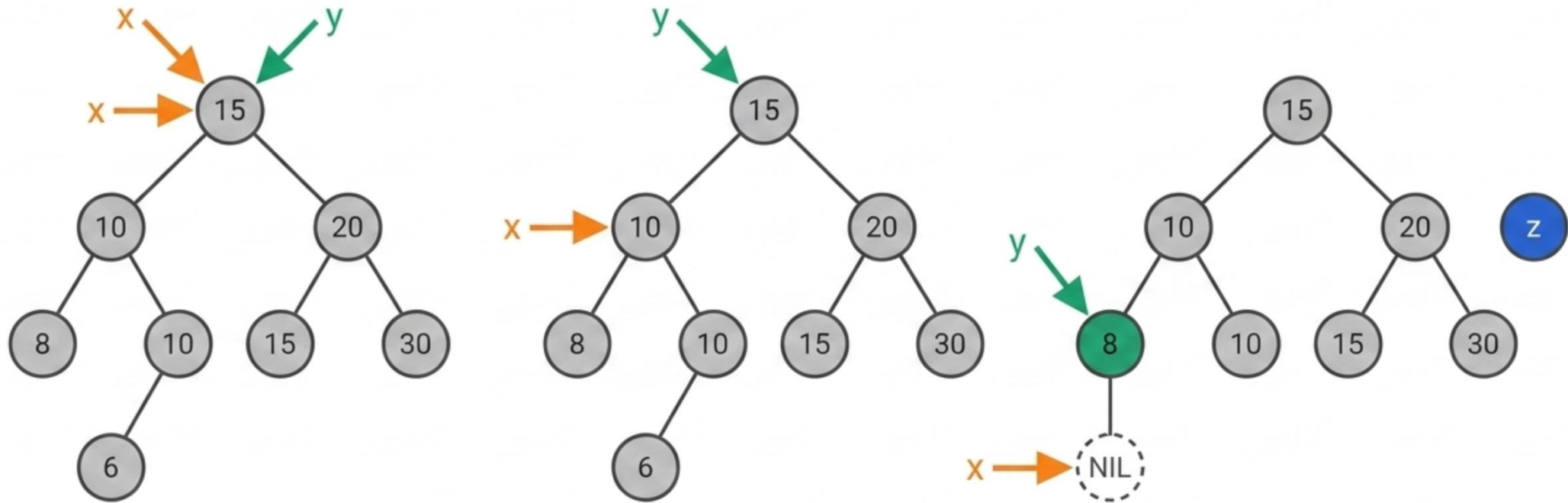
```
// 2. Descenso para aislar la posición
while (x != nullptr) {
    y = x;
    if (k < x->key) {
        x = x->left.get();
    } else if (x->key < k) {
        x = x->right.get();
    } else {
        return; // Abortar mutación (RAII libera 'z')
    }
}
```

El puntero y captura la referencia de x justo antes de que x descienda. Si k es idéntico, la función retorna inmediatamente protegiendo el conjunto.



Operaciones de mutación

Al finalizar el ciclo, la incertidumbre desaparece. x ha encontrado el vacío topológico exacto, e y retiene de forma segura el nodo que se convertirá en el padre anatómico.

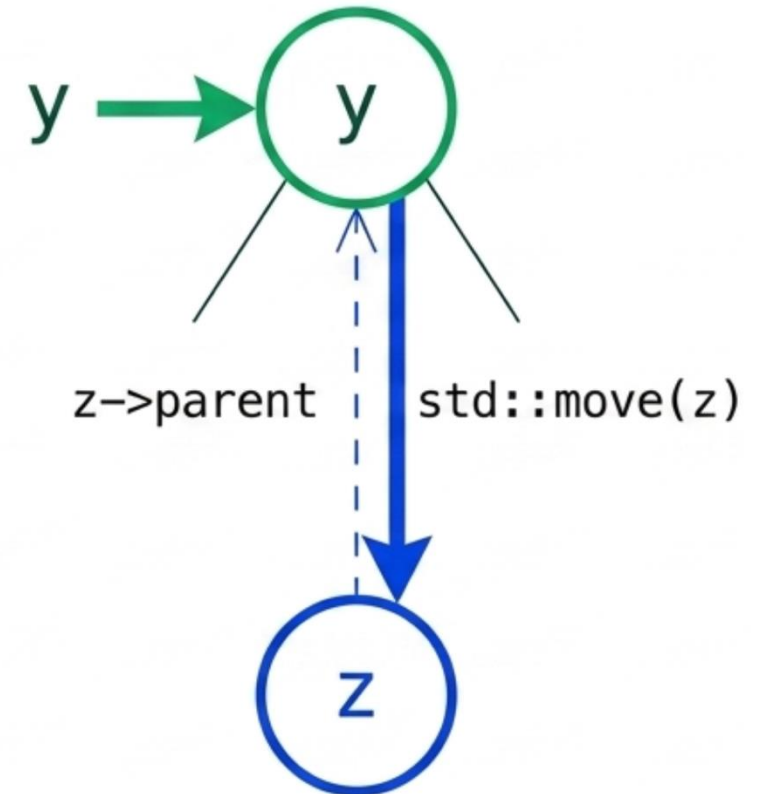


Operaciones de mutación

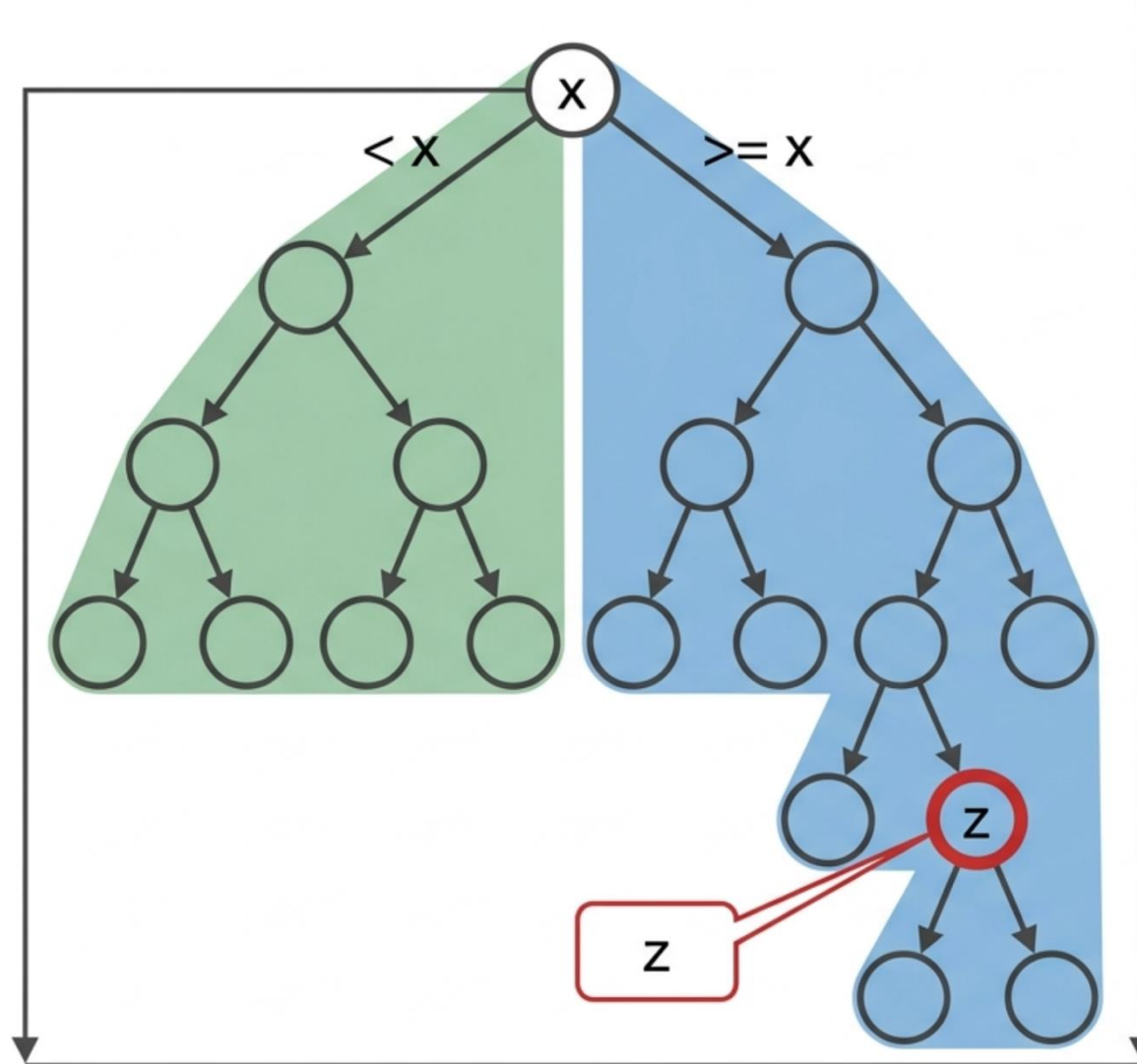
```
// 3. Enlace ascendente (back-pointer)
z->parent = y;

// 4. Mutación y Transferencia (Ownership Transfer)
if (y == nullptr) {
    root = std::move(z);
} else if (z->key < y->key) {
    y->left = std::move(z);
} else {
    y->right = std::move(z);
}
```

***std::move** invalida el puntero local z y transfiere el control absoluto de la memoria a la estructura persistente del árbol.*



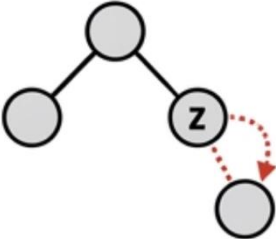
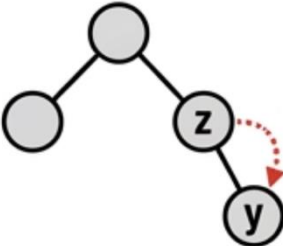
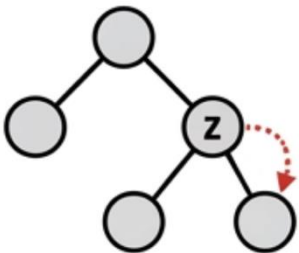
Operaciones de mutación



La Invariante y el Problema

- Todo nodo y en el subárbol izquierdo de x cumple:
 $y.key < x.key$.
- Todo nodo y en el subárbol derecho de x cumple:
 $y.key \geq x.key$.
- **El Objetivo:** Extraer el nodo z preservando la **propiedad estructural** en tiempo $O(h)$.

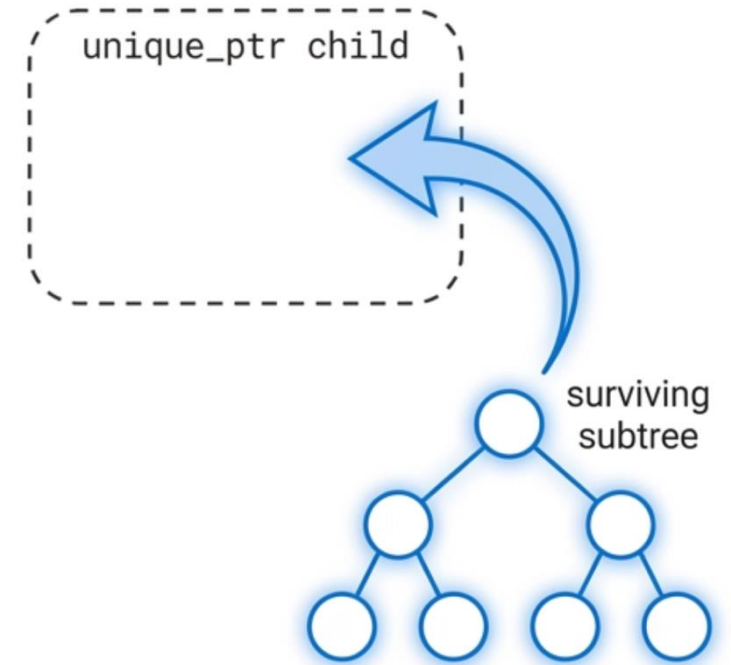
Operaciones de mutación

Aridad (Hijos)	Grafo Topológico	Complejidad	Solución Algorítmica
0 Hijos		Trivial	Desconectar y destruir z.
1 Hijo		Baja	'Elevar' al único hijo para reemplazar a z.
2 Hijos		Alta	Requiere cirugía mayor. Aislar al sucesor in-order.

Implementación (CASO 1)

```
if (z->left == nullptr || z->right == nullptr) {  
    std::unique_ptr<BSTNode<T>> child = nullptr;  
    if (z->left != nullptr) {  
        child = std::move(z->left);  
    } else if (z->right != nullptr) {  
        child = std::move(z->right);  
    }  
}
```

Lógica: Determinar el hijo sobreviviente (si existe) y capturar su propiedad mediante `std::move`.

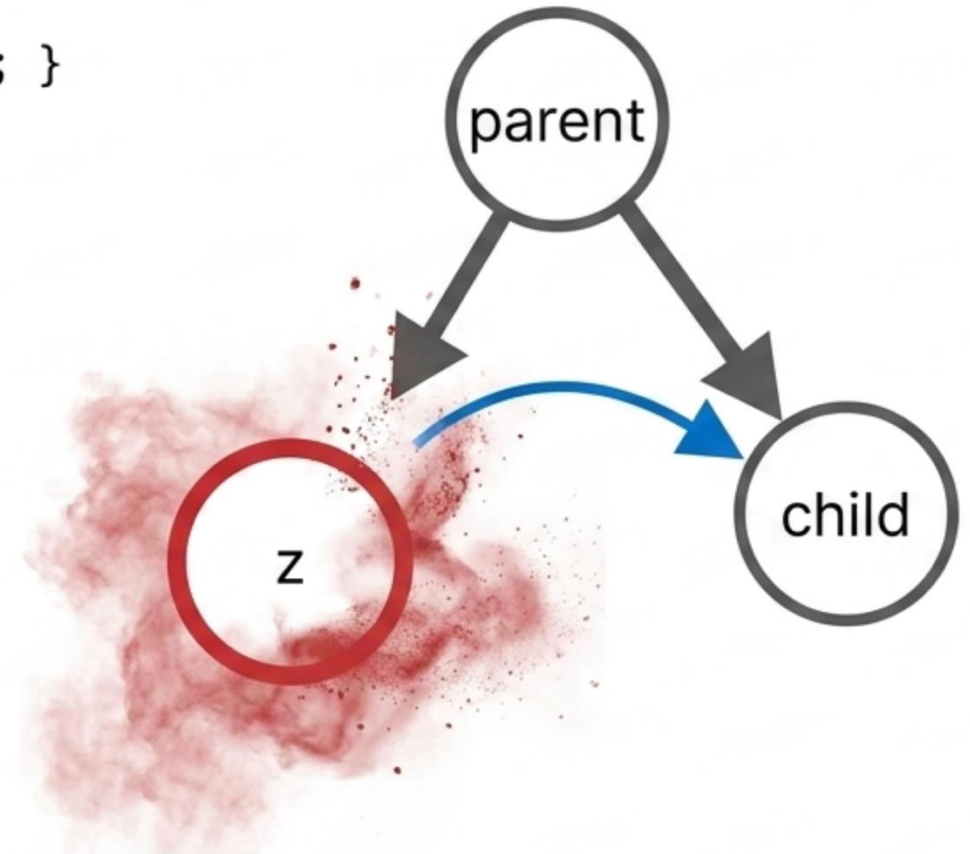


Implementación (CASO 2)

```
if (child != nullptr) { child->parent = z->parent; }  
  
if (z->parent == nullptr) {  
    root = std::move(child);  
} else if (z == z->parent->left.get()) {  
    z->parent->left = std::move(child);  
} else {  
    z->parent->right = std::move(child);  
}
```

Reasignación: El padre transfiere su propiedad al child sobreviviente.

Destrucción: z es destruido implícitamente por RAII al salir de alcance. Cero fugas de memoria.



Implementación (CASO 3)

```
const BSTNode<T>* const_y =  
tree_minimum(z->right.get());  
BSTNode<T>* y =  
const_cast<BSTNode<T>*>(const_y);  
  
z->key = std::move(y->key);  
  
tree_delete(root, y);
```

Paso 1: Encontrar sucesor
y en subárbol derecho.

Paso 2: std::move
sobrescribe la clave de z.
Invariante preservada.

Paso 3: Eliminación
recursiva de y.